Why JavaScript?

Console, Variables, Math & Data — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser** and **DevTools** open beside you. For every task, **attempt and write your answer *before* we discuss it** — your first guesses matter more than being right.

Part 1 — Why JavaScript?

TRY THIS · BROWSER

Open your `ecommerce-landing.html`. Click "Add to Cart" a few times. Click "Shop the Collection" too.

Q1. Does the **cart count** change? Does anything actually happen, besides the hover/active color changes you added? What's missing?

Q2. HTML gives the page **structure**, CSS gives it **style** and even simple hover feedback — but what's missing? What would you need in order to actually track how many items are in the cart, or make the page respond differently depending on what the user does?

Part 2 — The Console: Your Window Into JavaScript

TRY THIS · CONSOLE

Open DevTools → Console tab. Type `5 + 3` and press Enter.

Q3. What showed up? Did it show the text "5 + 3", or the calculated result? The console evaluates whatever you type and shows you the answer — no special command needed, yet.

Part 3 — Variables: Storing Values

Typing `5 + 3` directly works once — but what if you need to reuse a value many times, like a cart total that keeps changing?

TRY THIS · CONSOLE

```
price = 10;
price
```

Q4. What showed up in the console? What do you think `price = 10;` just did?

TRY THIS · CONSOLE

```
price = 20;
price
```

Q5. What changed? Could you reuse the same name, `price`, and give it a different value?

TRY THIS · CONSOLE

```
discount = 12
tax = 0.18;
```

Q6. The first line doesn't end with `;` but the second one ends with `;`. Did both lines still work? Should you prefer to use semicolon in your code? Why?

Q7. In your own words, why is it useful to store a value under a name like `price`, instead of typing the number `10` every time you need it? *(We'll get into the exact rules for declaring variables properly in Part 7.)*

Part 4 — Doing Math with Variables

TRY THIS · CONSOLE

```
10 + 3
10 - 3
10 * 3
10 / 3
```

One at a time, to see each result.

Q8. Run each line one at a time. Which result was **different from what you expected**, and why do you think JavaScript gave that answer?

TRY THIS · CONSOLE

Try `10%3`.

Q9. What did `%` produce? Any guess what this operator is called and what it's calculating?

Part 5 — Datatypes

So far every variable held a number. But what about a name, or a true/false answer?

TRY THIS · CONSOLE

```
typeof 'Alex'
typeof 21
typeof true
```

Q10. What did `typeof` print for each? These are called **primitive datatypes**. Any guess why they'd be called "primitive"?

How many datatypes are there in JavaScript?
JavaScript has **8 datatypes**: 7 primitive and 1 non-primitive.
Primitives (7): `string`, `number`, `boolean`, `undefined`, `null`, `bigint`, `symbol`.
Non-Primitive (1): `object` — we will talk in depth about it in future sessions.

Q11. Which of these do you think you'll use **most often** when building a real web page?

Q12. In your own words: what's a **primitive datatype**?

Part 6 — Moving Out of the Console: <script> and src

The console disappears the moment you refresh, and nobody else can run your code. What if it ran automatically instead — and with no console to type into, how would you make anything show up?

TRY THIS · VS CODE

Create a fresh `index.html` with the usual boilerplate. Right before the closing `</body>` tag, add `<script>console.log("Hello from the page!");</script>` directly inline. Open it in the browser and check the console.

Q13. Did the message show up just from opening the page — without you typing anything into the console? Where do you think this code actually ran?

TRY THIS · VS CODE

Create a separate file, `script.js`, containing `console.log("Hello from script.js!");`. Remove the inline `<script>` and replace it with `<script src="script.js"></script>`.

Q14. Did the message still show up? What do you think `src` is doing here — does this remind you of anything from CSS?

Q15. Why might keeping JavaScript in its own **external file** (just like external CSS) be better than writing it inline, especially as a project grows?

Part 7 — Declaration Types: var, let, const

*Back in Part 3, you gave a value a name with plain `price = 10;` — no keyword at all. That actually works in the console, but it's not really how you're supposed to declare a variable. JavaScript gives you three proper ways to do it: `var`, `let`, and `const`. `var` is the oldest, and it still shows up in older code and tutorials, so it's worth knowing why we've mostly **moved away** from `var` in favor of `let` and `const`.*

TRY THIS · CONSOLE

```
var x = 5;
console.log(x);

var x = 10;
console.log(x);
```

Q16. Did redeclaring `var x` cause an error? What happened to the value?

TRY THIS · CONSOLE

```
let y = 5;
let y = 10;
```

Q17. What happened this time — error or no error? How does `let` behave differently from `var` when you try to redeclare it?

TRY THIS · CONSOLE

```
if (true) {
  var a = "I'm var";
  let b = "I'm let";
  console.log("inside", a);
  console.log("inside", b);
}
console.log("outside", a);
console.log("outside", b);
```

Q18. Both variables printed fine *inside* the `if` block. But outside it, which one still printed and which one gave an error? What does this tell you about where `var` "lives" versus where `let` "lives"?

TRY THIS · CONSOLE

```
console.log(x);
var x = 5;
```

Q19. What did this do — did `var x` print something, or throw an error? Was that what you expected, given that `console.log` runs *before* the line that creates `x`?

TRY THIS · CONSOLE

Now comment all the earlier code, then type:

```
console.log(y);
let y = 5;
```

Q20. What happened with `let y` this time? `let` and `const` variables can't be touched before their declaration line runs — this is called an "access before initialization" error. Why do you think `var` behaves so differently here?

Q21. In your own words, why do modern style guides recommend avoiding `var` in favor of `let` and `const`? When would you personally reach for `let` vs `const`?

Concept — Naming Your Variables. Stick to **camelCase** for variable names — start lowercase, then capitalize each new word: `productPrice`, `isLoggedIn`, `cartTotal` (never `product_price` or `ProductPrice`). And always pick a **meaningful name** that describes what the value actually is — `cartTotal` tells you far more than `x` or `temp1` ever could.

Default to const. It might sound restrictive, but in real production code, developers reach for `const` by default for **every** variable — and only switch to `let` if that specific variable actually needs to be reassigned later. Seeing `let` in someone's code immediately signals "this value changes," while `const` signals "this value is locked in."

Part 8 — Strings: Concatenation with + and Template Literals

You've logged numbers and single strings — but what if you need to build a message out of several pieces, like a variable dropped into a sentence?

TRY THIS · CONSOLE

```
let firstName = 'Nikhil';
let lastName = 'Bhatt';
console.log(firstName + lastName);
```

Q22. What did you get? Is there a space between the two names? What do you think `+` does when used with strings, compared to when it's used with numbers?

Concept — Concatenation. Joining strings together end-to-end with `+` is called **concatenation**. JavaScript glues them exactly as written — it does *not* add spaces, commas, or punctuation for you. Anything you want between the pieces, you have to put there yourself.

TRY THIS · CONSOLE

```
console.log(firstName + ' ' + lastName);
console.log(firstName, lastName);
```

Q23. Both lines put a space between the names, but they got there differently. In the first, what did **you** have to add by hand? In the second, who added the space — you, or `console.log` itself?

TRY THIS · CONSOLE

```
let price = 28;
console.log('Total: ' + price + 5);
```

Q24. What did you get — "Total: 33", or something else? If it wasn't 33, any guess why? *(Hint: JavaScript reads left to right.)*

TRY THIS · CONSOLE

Now try `console.log(price + 5 + ' total');` — numbers first, string last.

Q25. What did you get this time? Why does the order of `+` change the result?

Manually gluing pieces together with `+`, tracking spaces and punctuation, gets messy fast — especially with several variables. What if there were a cleaner way to drop variables straight into a sentence?

Concept — Template Literals. Instead of quotes, you can wrap a string in **backticks** (`` ` `` — the key just above Tab). Inside backticks, you can drop a variable straight into the sentence by writing it as `${variableName}`. JavaScript swaps in the value, and you type the spaces and punctuation exactly where you want them — no `+` needed. A string written this way is called a **template literal**.

TRY THIS · CONSOLE

```
let name = 'Chaitanya';
let item = 'coffee';
console.log(`Hello ${name}, your ${item} is ready!`);
```

Q26. What did this print? In your own words, what's the purpose of `${}` — what does JavaScript do with whatever you put inside it?

Q27. Revisit **Q24** — now that you've seen template literals, use it and see if it helps you get the correct total.

Q28. In your own words: what's the difference between string concatenation with `+` and **template literals**? Which would you reach for, and why?

Part 9 — Conditionals: ===, !==, and the Confusing ==, !=

So far you've only stored and displayed values. But real programs need to make decisions — like "if the cart has items, show the checkout button." That means comparing values and asking true/false questions.

TRY THIS · VS CODE

Back in your `script.js`, add these two lines, then refresh the page and check the console:

```
console.log(5 === 5);
console.log(5 === '5');
```

Q29. Why might `5 === '5'` not be considered equal?

TRY THIS · CONSOLE

```
console.log(5 == '5');
```

Q30. What did this return, compared to `===`? What's different about `==` that makes it treat `5` and `'5'` as equal?

TRY THIS · CONSOLE

```
console.log(5 !== '5');
console.log(5 != '5');
```

Q31. What did each return? Based on everything so far, what do you think `!==` and `!=` check for?

TRY THIS · BROWSER

Google search "**MDN docs loose equality**" and open the MDN page on equality comparisons. Jump to the section where the **rules** are written, and read through the rules for how `==` decides two values are equal.

Q32. In your own words, summarize the rules MDN lists for `==` (loose equality) — when does it consider two different-looking values equal?

TRY THIS · BROWSER

Now Google search "**MDN docs strict equality**" and open that MDN page too. Read through its rules.

Q33. Summarize the rules for `===` (strict equality). How are they different from — or simpler than — the `==` rules you just read? Based on both, which one would you recommend using by default in your own code, and why do you think professional codebases almost always pick one over the other?

CLASS WRAP-UP

Push all this code to GitHub